

Il file `catalogo.csv` contiene il catalogo dei libri disponibili in una libreria. Di seguito si riporta un estratto del file.

1: File `catalogo.csv`

```
autore,titolo,genere,media_recensioni,numero_recensioni
Francesca Altavento,La firma sulla busta rossa,Giallo,4.9,70
Beatrice Nottelari,La scomparsa del notaio,Giallo,2.7,260
Laura Pietradoro,La bottega dei ricordi,Narrativa,1.7,350
Valentina Torrelume,La domenica dei girasoli,Narrativa,1.3,180
Luca Rivadoro,Le campane della citta assediata,Storico,3.6,425
Giulia Monteluca,La rivolta dei tintori,Storico,3.6,220
Simone Chiaravento,La chiave della stanza sette,Giallo,2.5,190
Nicola Portasole,Le lettere del giovedi,Narrativa,3.7,20
Federica Ondaviva,La notte dei cinque sospetti,Giallo,1.1,450
Francesca Altavento,Tre indizi sul pianerottolo,Giallo,2.7,20
...
```

Il contenuto del file può essere interpretato in base alla linea di intestazione, che definisce le caratteristiche con cui è descritto ogni libro: `autore`, `titolo`, `genere`, `media_recensioni`, `numero_recensioni`. Si assuma che i valori di `numero_recensioni` siano strettamente positivi e i valori di `media_recensioni` siano compresi fra 1 e 5.

La libreria desidera predisporre una vetrina selezionando, per ciascun genere, al più tre libri di autori diversi, privilegiando quelli con la valutazione media più alta. Considerando esclusivamente le dieci righe riportate sopra, una possibile vetrina è rappresentata nella lista seguente.

```
[
  ("Francesca Altavento", "La firma sulla busta rossa", "Giallo", 4.9, 70),
  ("Beatrice Nottelari", "La scomparsa del notaio", "Giallo", 2.7, 260),
  ("Simone Chiaravento", "La chiave della stanza sette", "Giallo", 2.5, 190),
  ("Nicola Portasole", "Le lettere del giovedi", "Narrativa", 3.7, 20),
  ("Laura Pietradoro", "La bottega dei ricordi", "Narrativa", 1.7, 350),
  ("Valentina Torrelume", "La domenica dei girasoli", "Narrativa", 1.3, 180),
  ("Luca Rivadoro", "Le campane della citta assediata", "Storico", 3.6, 425),
  ("Giulia Monteluca", "La rivolta dei tintori", "Storico", 3.6, 220)
]
```

Il libro *Tre indizi sul pianerottolo* non viene selezionato perché, per il genere Giallo, è già presente un libro della stessa autrice (*Francesca Altavento*) con valutazione maggiore.

1. Definire la funzione `leggi_catalogo`

- **Parametri di ingresso:**
 - `percorso_file`: stringa
- **Restituisce:** una lista di tuple, ciascuna costituita da tre stringhe e due numerici.
- **Descrizione:** la funzione apre il file il cui percorso è fornito nel parametro di ingresso, salta la riga di intestazione, e costruisce per ciascuna delle righe rimanenti una tupla contenente i valori riportati nella riga, convertendo opportunamente i valori numerici. Restituisce la lista di tali tuple.
- **Esempio:** se la funzione viene chiamata con il percorso del file `catalogo.csv` come argomento, essa restituisce la seguente lista (troncata per ragioni di spazio):

```
[("Francesca Altavento","La firma sulla busta rossa","Giallo",4.9,70),  
 ("Beatrice Nottelari","La scomparsa del notaio","Giallo",2.7,260), ... ]
```

2. Definire la funzione `riepilogo_generi`• **Parametri di ingresso:**

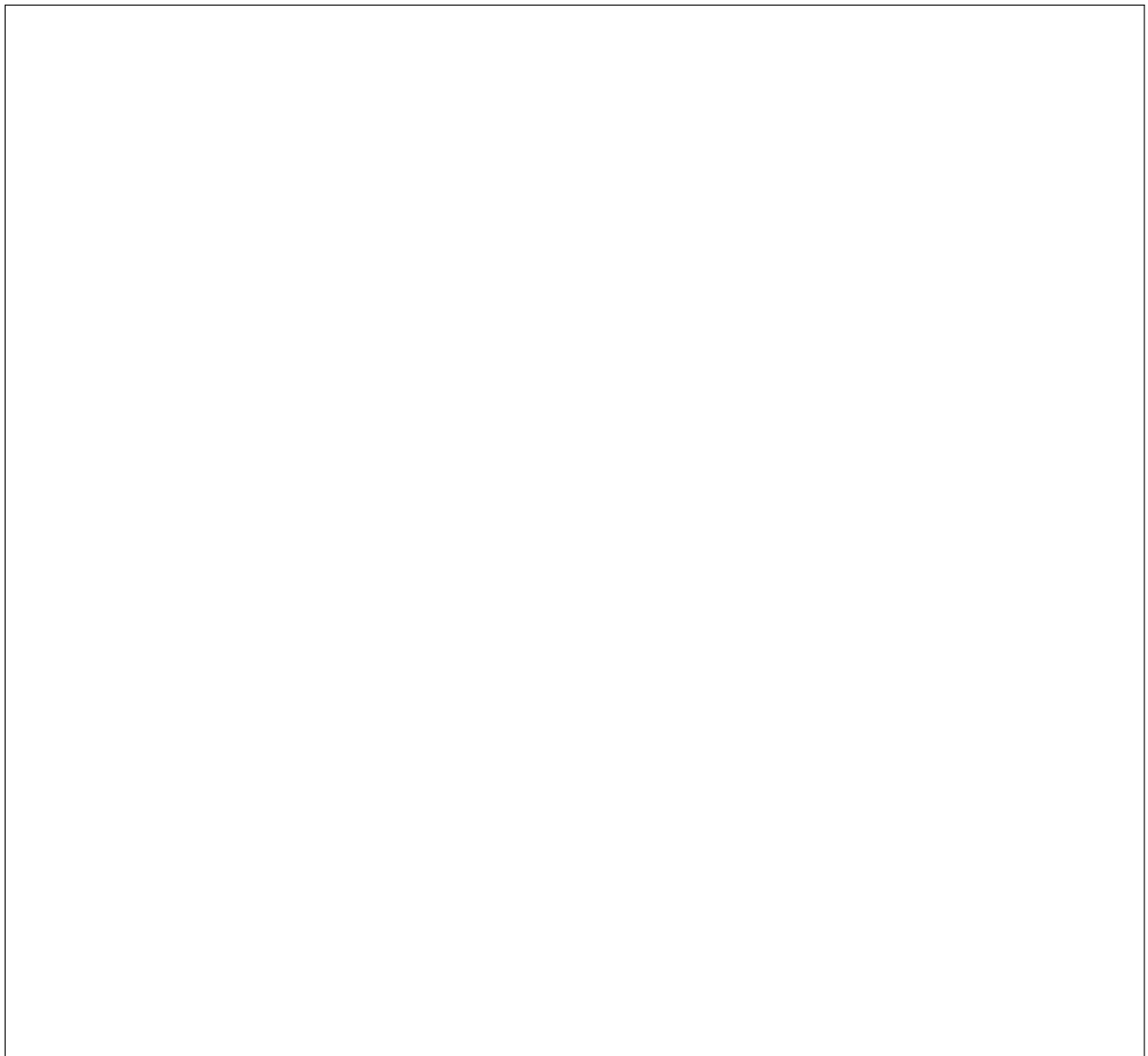
- `catalogo`: lista di tuple, ciascuna nel formato (`autore`, `titolo`, `genere`, `media_recensioni`, `numero_recensioni`)

• **Restituisce:** una lista di tuple, ciascuna contenente una stringa ed un float.• **Descrizione:** la funzione calcola, per ciascun valore distinto di `genere` delle tuple in `catalogo`, la media pesata dei valori di `media_recensioni` delle tuple in `catalogo` aventi tale valore di `genere`, utilizzando come pesi i corrispondenti valori di `numero_recensioni`. La funzione restituisce una lista di tuple contenenti ciascuna il genere e la media calcolata. L'ordine delle tuple non è rilevante.• **Esempio:** dato il seguente valore di `catalogo`:

```
[("a", "b", "X", 4.0, 10),  
 ("c", "d", "Y", 3.0, 20),  
 ("e", "f", "X", 2.0, 30)]
```

la funzione restituisce:

```
[("Y", 3.0), ("X", 2.5)]
```



3. Definire la funzione `filtra_catalogo`• **Parametri di ingresso:**

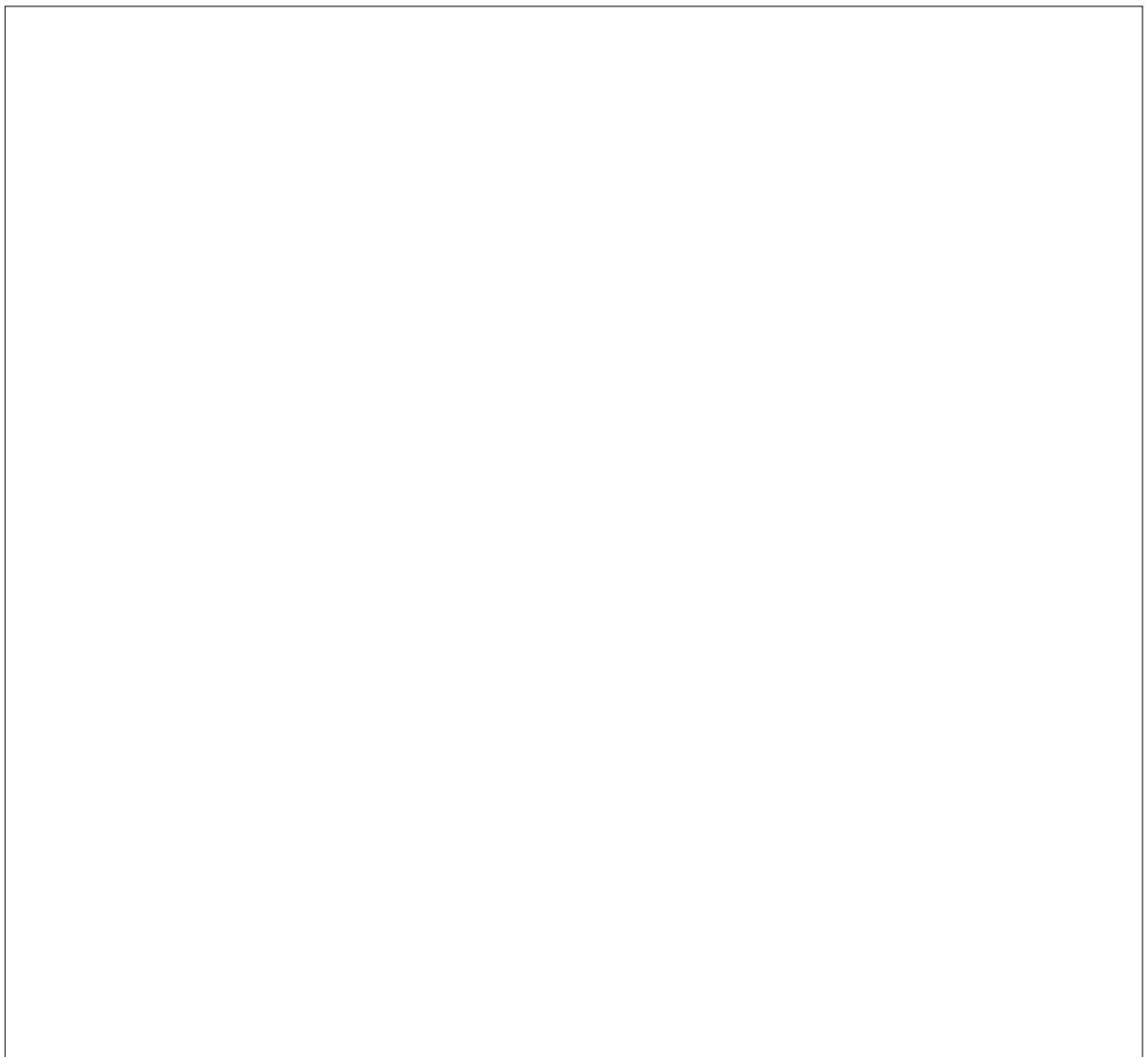
- `catalogo`: lista di tuple, ciascuna nel formato (`autore`, `titolo`, `genere`, `media_recensioni`, `numero_recensioni`)
- `genere_richiesto`: stringa

• **Restituisce:** una lista di tuple• **Descrizione:** la funzione restituisce una lista contenente le tuple di `catalogo` il cui valore di `genere` coincide con `genere_richiesto`, ordinate in ordine decrescente rispetto al valore di `media_recensioni`.• **Esempio:** dato `genere_richiesto = "X"` e il seguente valore di `catalogo`:

```
[("a", "b", "X", 4.0, 10),  
 ("c", "d", "Y", 3.0, 20),  
 ("e", "f", "X", 2.0, 30)]
```

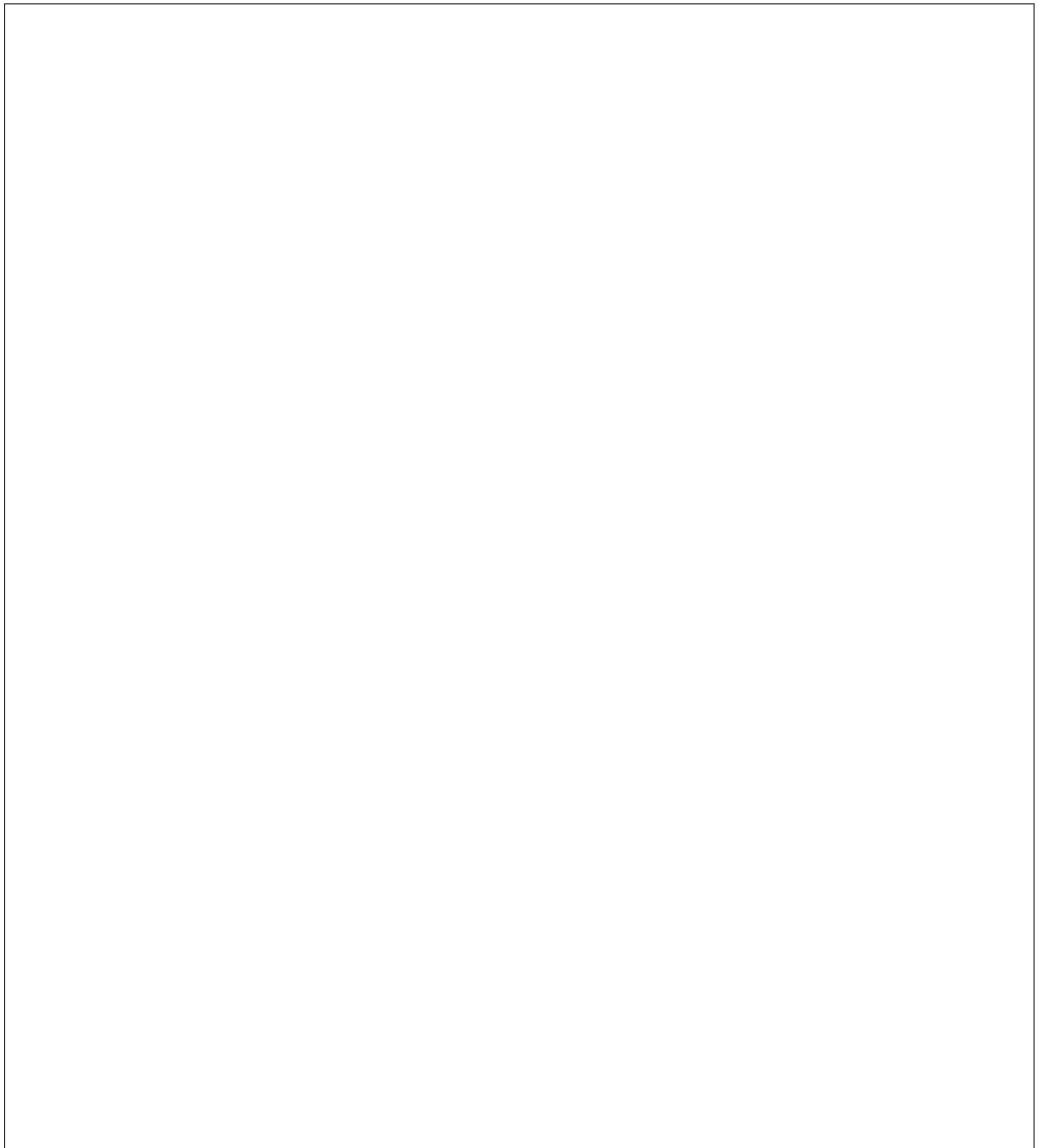
la funzione restituisce:

```
[("a", "b", "X", 4.0, 10),  
 ("e", "f", "X", 2.0, 30)]
```



4. Definire la funzione `crea_vetrina`

- **Parametri di ingresso:**
 - `catalogo`: lista di tuple, ciascuna nel formato (`autore`, `titolo`, `genere`, `media_recensioni`, `numero_recensioni`)
- **Restituisce:** una lista di tuple
- **Descrizione:** la funzione ricava i generi presenti in `catalogo` usando `riepilogo_generi`. Per ciascun genere, ottiene la lista ordinata delle relative tuple mediante `filtra_catalogo` e seleziona al più tre tuple con valori di `autore` distinti tra loro e con i valori più alti di `media_recensioni`. Restituisce la lista delle tuple selezionate.
- **Esempio:** si faccia riferimento all'esempio riportato nella pagina introduttiva.



Legenda

s, **s1**: stringa **a**, **b**: numero **i**, **j**, **k**, **n**: intero **x**: elemento generico
l, **l1**: lista **d**, **d1**: dizionario **r**, **r1**: set **t**, **t1**: tupla

Principali funzioni built-in (ordine alfabetico)

abs(a): restituisce il valore assoluto di un numero
enumerate(iterable): restituisce un iteratore i cui elementi sono tuple contenenti il conteggio e il valore ottenuto dall'iterazione su iterable
input(s): scrive il prompt **s** sullo standard output e restituisce stringa acquisita
instance(object, classinfo): restituisce **True** se **object** è del tipo **classinfo** o di una sottoclasse di **classinfo**. Altrimenti, restituisce **False**
len(x): restituisce la lunghezza (numero di elementi) di **x**. **x** può essere una sequenza (ad es.: string, list, tuple, range) o un contenitore (ad es.: dict, set)
max(iterable, *, key=None), max(arg1, arg2, *args, key=None): restituisce il massimo nell'iterabile, o il massimo tra due o più argomenti. La funzione **key** serve a calcolare il valore da usare per confrontare gli elementi
min: analogo a **max**

print(*x, sep=' ', end='\n'): stampa oggetti, separati da **sep** e seguiti infine da **end**
range(j), **range(i, j, k)**: restituisce una sequenza immutabile per progressione aritmetica
reversed(seq): restituisce un iteratore che percorre **seq** in ordine inverso
round(a, n): arrotonda il valore di **a** all'intero più vicino o ad **n** cifre decimali
sorted(iterable, key=None, reverse=False): restituisce una nuova lista ordinata degli elementi di iterabile. La funzione **key** serve a calcolare il valore da usare per confrontare gli elementi. **reverse=True** inverte l'ordine
sum(iterable): restituisce la somma dei valori dell'iterabile
zip(*iterables): itera in parallelo su diversi iterabili e restituisce un iteratore costituito da tuple. La *i*-esima tupla è costituita dall'*i*-esimo elemento di ciascuno degli iterabili

Indexing e Slicing

seq[i]: restituisce l'elemento di indice **i** della sequenza
seq[i:j]: restituisce una sottosequenza di **seq** dall'indice **i** all'indice **j** (escluso)
seq[i:j:k]: restituisce una sottosequenza di **seq** da indice **i** a indice **j** (escluso) con step **k**

Modulo Random

choice(seq): restituisce un elemento casuale dalla sequenza **seq**
choices(seq, k=1): restituisce una lista di **k** elementi scelti da **seq** con reinserimento
random(): restituisce un float casuale in $[0,1)$
randint(i, j): restituisce un intero casuale tra **i** e **j** (inclusi)
sample(seq, k): restituisce una lista di **k** elementi unici scelti da **seq** senza reinserimento
shuffle(seq): rimescolamento della sequenza **seq** in-place
uniform(a, b): restituisce un float casuale tra **a** e **b** (inclusi)

Principali funzioni del modulo math

math.cos(a), **math.sin(a)**, **math.exp(a)**, **math.log(a)**, **math.log2(a)**, **math.sqrt(a)**

Principali eccezioni

FileNotFoundError, **IndexError**, **KeyError**, **ValueError**, **Exception** (eccezione generica)

Stringhe e principali metodi

Trasformazioni (non in-place): **s.lower()**, **s.upper()**, **s.replace(s1,s2)**, **s.strip()**
Controlli: **s.startswith(s1)**, **s.endswith(s1)**, **s.islower()**, **s.isupper()**, **s.isdigit()**
Da lista a stringa: **s.join(l1)**, Da stringa a lista: **s.split(sep)**
Altro: **s.count(s1)**, **s.index(s1)**; Conversione: a intero **int(s)**, a float **float(s)**
Concatenazione: **s1 + s2** Replicazione: **s1 * n** Membership: **s in s1**

Liste e principali metodi

Modifiche (in-place): **l.append(x)**, **l.extend(l1)**, **l.insert(i, x)**, **l.pop(i)**, **l.remove(x)**,
l.reverse(), **l.sort(key=None, reverse=False)**
Altro: **l.count(x)**, **l.index(x)**
List comprehension: **l = [expression for item in iterable]**
Concatenazione: **l1 + l2** Replicazione: **l1 * n** Membership: **x in l**

Tuple e principali metodi

Altro: **t.count(x)**, **t.index(x)**
Concatenazione: **t1 + t2** Replicazione: **t1 * n** Membership: **x in t**

Insiemi e principali metodi

Modifiche (in-place): **r.add(x)**, **r.remove(x)**, **r.discard(x)**, **r.clear()**
Operazioni: **r.difference(r1)** (oppure **r - r1**), **r.symmetric_difference(r1)** (oppure **r^r1**),
r.union(r1) (oppure **r|r1**), **r.intersection(r1)** (oppure **r&r1**)
Controlli: **r.issubset(r1)**, **r.issuperset(r1)**, **r.isdisjoint(r1)**
Membership: **x in r1**

Dizionari e principali metodi

Modifiche (in-place): **d1.update(d2)**, **d.pop(key)**
Viste: **d.keys()**, **d.values()**, **d.items()**
Accesso: **d[key]**, **d.get(key, x=None)**
Aggiunta o modifica: **d[key] = val**
Membership: **key in d1**
Dict comprehension: **d = {key:val for item in iterable}**

f-string

Esempio numeri (cifre decimali): **f"{3.14:.1f}"** → **'3.1'**
Esempio stringa con allineamento (<, >, ^): **f"{'ciao':<10}"** → **'ciao.....'**
f"{'Trieste':<10} {10:>3}" → **'Trieste... 10'**
f"{'Udine':<10} {7:>3}" → **'Udine..... 7'**

File

Apertura: **open(file, mode, encoding)** apre un file e restituisce un file object **f**
Chiusura: **f.close()**; automatica se il file è aperto con il **with** statement
Lettura: **f.read()** (restituisce una stringa con l'intero contenuto del file), **f.readline()** (restituisce una stringa, fino al prossimo **\n**), **f.readlines()** (restituisce una lista di stringhe)
Scrittura: **f.write(s)**, **f.writelines(l)**